

SQL

consultas

Carina F. Dorneles
dorneles@inf.ufsc.br

Parte I

SQL

- **DDL:** possui comandos para *implementar* o modelo lógico
 - Comando para **criação** das tabelas
 - Comando para **alteração** das tabelas
 - Comando para **remoção** das tabelas
 - **DML:** possui comandos para *manipular* os dados armazenados no Banco de Dados
 - Comando para **inserção** de dados nas tabelas
 - Comando para **alteração** de dados nas tabelas
 - Comando para **remoção** de dados nas tabelas
 - **Comando para consulta** aos dados nas tabelas
-

SQL

- **DDL:** possui comandos para *implementar* o modelo lógico
 - Comando para **criação** das tabelas
 - Comando para **alteração** das tabelas
 - Comando para **remoção** das tabelas
 - **DML:** possui comandos para *manipular* os dados armazenados no Banco de Dados
 - Comando para **inserção** de dados nas tabelas
 - Comando para **alteração** de dados nas tabelas
 - Comando para **remoção** de dados nas tabelas
 - **Comando para consulta** aos dados nas tabelas
-

Consulta a dados no SQL

- Padrão
 - SQL ANSI 92
 - SQL3 - SQL:2003: inclui cláusulas para manipular XML
 - Extensões (algumas)
 - Transact-SQL – SQL Server
 - PLpg/SQL – PostgreSQL
 - PL/SQL – Oracle
 - SQL PL – IBM
 - ...
-

Modelo de Consulta SQL – SQL92

SELECT <Lista de colunas, função agregação>

FROM <Lista de tabelas>

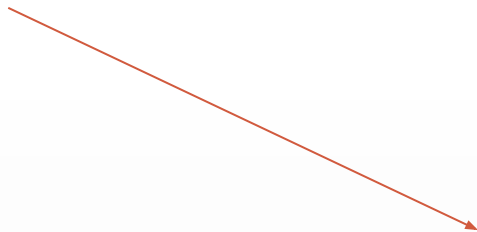
[WHERE <Condição>**]**

Modelo de Consulta SQL

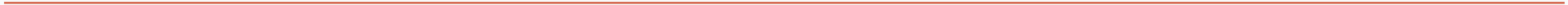
SELECT <Lista de colunas, função agregação>

FROM <Lista de tabelas>

[WHERE <Condição>]



Cláusula `WHERE` é opcional



Banco de dados Exemplo

Livro

<u>Codigo</u>	Título	Ano	NrPaginas
LI005	Web e Banco de dados	2013	330
LI670	Introdução a Banco de Dados	2000	500
LI340	Programação C	2012	250
LI003	Algoritmos e Lógica	2000	700

Pessoa

<u>Codigo</u>	Nome	Idade	fone	CodEsposa	Sexo
PE02	Aninha	23	9999.9999	NULL	F
PE10	Paulinho	20	8888.8888	NULL	M
PE87	Juca	34	7777.7777	PE02	M
PE23	Luana	30	6666.6666	NULL	F
PE54	Beto	28	5555.5555	PE23	M



Empréstimo

<u>CodLivro</u>	<u>CodPessoa</u>	<u>Data</u>	<u>Hora</u>
LI005	PE02	10/10/2000	8:00
LI670	PE02	10/10/2000	8:00
LI340	PE23	01/11/2000	11:50
LI003	PE54	20/11/2000	10:00
LI005	PE10	11/11/2000	14:00
LI670	PE87	23/05/2001	16:15

Livro

<u>Codigo</u>	Título	Ano	NrPaginas
LI005	Web e Banco de dados	2013	330
LI670	Introdução a Banco de Dados	2000	500
LI340	Programação C	2012	250
LI003	Algoritmos e Lógica	2000	700

Voltando um pouco na álgebra.....

- Q1: Recuperar o título e ano dos livros com mais de 350 páginas

Voltando um pouco na álgebra.....

- Q1: Recuperar o título e ano dos livros com mais de 350 páginas

π titulo, ano (σ nrPaginas > 350 (Livro))

Livro

<u>Codigo</u>	Título	Ano	NrPaginas
LI005	Web e Banco de dados	2013	330
LI670	Introdução a Banco de Dados	2000	500
LI340	Programação C	2012	250
LI003	Algoritmos e Lógica	2000	700

Execução no Banco de dados



A árvore de consulta é executada de baixo para cima

- primeiro **acessa a tabela**
- depois executa **seleção**
- em seguida a **projeção**

Seleção sempre é executada antes da **projeção**

Árvore algébrica vs. SQL

π título, ano



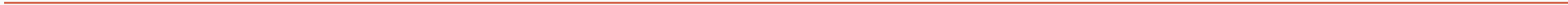
σ nrPaginas > 350



1º



FROM <Lista de tabelas>



Árvore algébrica vs. SQL



Árvore algébrica vs. SQL



No SQL

- Q1: Recuperar o título e ano dos livros com mais de 350 páginas

π titulo, ano (σ nrPaginas > 350 (Livro))

No SQL

- Q1: Recuperar o título e ano dos livros com mais de 350 páginas

π titulo, ano (σ nrPaginas > 350 (Livro))

```
SELECT titulo, ano
FROM livro
WHERE nrPaginas > 350
```

No SQL

- Q1: Recuperar o título e ano dos livros com mais de 350 páginas

π titulo, ano (σ nrPaginas > 350 (Livro))

```
SELECT titulo, ano
FROM livro
WHERE nrPaginas > 350
```

No SQL

- Q1: Recuperar o título e ano dos livros com mais de 350 páginas

π titulo, ano (σ nrPaginas > 350 (Livro))

```
SELECT titulo, ano  
FROM livro  
WHERE nrPaginas > 350
```

Exemplos

“Recuperar **todos os atributos** dos pacientes, com **email ap@a.d.br**”

```
SELECT *  
FROM paciente  
WHERE email = 'ap@a.d.br'
```

```
WHERE nome = 'João' AND email = 'joao@ad.br'
```

“Recuperar nome, endereço e email dos pacientes cuja profissão seja diferente de ‘engenheiros’”

```
SELECT nome, endereco, email  
FROM paciente  
WHERE profissao <> 'engenheiro'
```

Exemplos

“Recuperar **todos os atributos** dos pacientes, com **email ap@a.d.br**”

```
SELECT *  
FROM paciente  
WHERE email = 'ap@a.d.br'
```

“Recuperar **o nome** dos pacientes, com **email ap@abc.de.br** e idade maior que 18

```
SELECT nome  
FROM paciente  
WHERE email = 'ap@abc.de.br' AND idade > 18
```

“Recuperar **nome, endereço e email** dos pacientes cuja profissão **seja diferente de engenheiro**”

```
SELECT nome, endereco, email  
FROM paciente  
WHERE profissao <> 'engenheiro'
```

Exemplos

“Recuperar **todos os atributos** dos pacientes, com **email ap@a.d.br**”

```
SELECT *  
FROM paciente  
WHERE email = 'ap@a.d.br'
```

“Recuperar **o nome** dos pacientes, com **email ap@abc.de.br** e idade maior que 18

```
SELECT nome  
FROM paciente  
WHERE email = 'ap@abc.de.br' AND idade > 18
```

“Recuperar **nome, endereço e email** dos pacientes cuja **profissão seja diferente de 'engenheiros'**”

```
SELECT nome, endereco, email  
FROM paciente  
WHERE profissao <> 'engenheiro'
```

Uso da expressão WHERE

- Operadores de comparação:

=, <>, <, <=, >, >=

<coluna> **BETWEEN** 'valorInicial' **AND** 'valorFinal'

<coluna> IN ('valor1', ..., 'valorn')

<coluna> LIKE '%banco de %'

<coluna> LIKE '19_ _'

<coluna> IS [NOT] NULL

"_" = um caracter

"%" = sequência de caracteres

- Operadores lógicos: AND e OR

Exemplos

- Recuperar todos os dados dos pacientes cujo nome **começa** com A

```
SELECT *  
FROM paciente  
WHERE nome LIKE 'A%'
```

- Recuperar dados dos paciente cujo nome **contenha** 'Silva'

```
SELECT *  
FROM paciente  
WHERE nome LIKE '%Silva%'
```

Exemplos

- Pacientes cuja idade esteja entre 20 e 30

```
SELECT *  
FROM paciente  
WHERE idade BETWEEN 20 and 30
```

OU WHERE idade >= 20 AND idade <=30

nunca "WHERE idade >= 20 AND <=30 /*ERRADO*/"

- Pacientes cuja idade esteja entre 20 e 30 e cuja cidade seja diferente de "Blumenau"

```
SELECT *  
FROM paciente  
WHERE (idade BETWEEN 20 and 30) AND cidade <> 'Blumenau'
```

Exemplos

- Pacientes cuja idade esteja entre 20 e 30

```
SELECT *  
FROM paciente  
WHERE idade BETWEEN 20 and 30
```

OU WHERE idade >= 20 AND idade <=30

nunca "WHERE idade >= 20 AND <=30 /*ERRADO*/"

- Pacientes cuja idade esteja entre 20 e 30 e cuja cidade seja 'Blumenau' ou 'Floripa'

```
SELECT *  
FROM paciente  
WHERE idade BETWEEN 20 AND 30 AND (cidade = 'Blumenau' OR cidade='Floripa')
```

Exemplos – uso do DISTINCT

- Selecionar a **idade** dos pacientes

```
SELECT idade  
FROM paciente
```

<i>idade</i>
10
10
10
70
70
70
70
20
...
...

- Selecionar a **idade** dos pacientes

```
SELECT DISTINCT idade  
FROM paciente
```

Exemplos – uso do DISTINCT

- Selecionar a **idade** dos pacientes

```
SELECT idade  
FROM paciente
```

<i>idade</i>
10
10
10
70
70
70
70
20
...
...

- Selecionar a **idade** dos pacientes

```
SELECT DISTINCT idade  
FROM paciente
```

<i>idade</i>
10
70
20

Juntando tabelas



Juntando tabelas - álgebra

- Produto cartesiano **X**
- Junção 



Juntando tabelas - álgebra

Q2: Recuperar título dos livros emprestados

- Produto cartesiano **X**

$\pi_{\text{Titulo}} (\sigma_{\text{codigo} = \text{codLivro}} (\text{Livro} \mathbf{X} \text{Emprestimo}))$

- Junção

$\pi_{\text{Titulo}} (\text{Livro} \bowtie_{\text{codigo} = \text{codLivro}} \text{Emprestimo})$

SQL

- Produto Cartesiano

π Titulo (σ codigo = codLivro (Livro **X** Emprestimo))

```
SELECT livro.titulo, emprestimo  
FROM livro, emprestimo  
WHERE codigo = codlivro
```



SQL

- Produto Cartesiano

π `Titulo` (σ `codigo = codLivro` (`Livro` **X** `Emprestimo`))

`FROM livro, emprestimo`

`WHERE codigo = codLivro`

SQL

- Produto Cartesiano

π Titulo (σ codigo = codLivro (Livro **X** Emprestimo))

FROM livro, ~~emprestimo~~

WHERE codigo = codLivro

SQL

- Produto Cartesiano

π `Titulo` (σ `codigo = codLivro` (`Livro` **X** `Emprestimo`))

```
SELECT titulo
FROM livro, emprestimo
WHERE codigo = codlivro
```

Juntando tabelas - álgebra

- Junção

π Titulo (Livro  codigo = codLivro Emprestimo)

FROM livro JOIN emprestimo ON codigo=codLivro

Juntando tabelas - álgebra

- Junção

π `Titulo` (`Livro`  `Emprestimo`)
`codigo = codLivro`

`FROM livro JOIN emprestimo ON codigo=codlivro`

Juntando tabelas - álgebra

- Junção



Juntando tabelas - álgebra

- Junção

π `Titulo` (`Livro`  `Emprestimo`)
`codigo = codLivro`

```
SELECT titulo
```

```
FROM livro JOIN emprestimo ON codigo=codlivro
```

Exemplos

- Recuperar título do livro e data de emprestimo, dos livros emprestados a partir de 2000

- Produto cartesiano

```
SELECT titulo, data  
FROM livro, emprestimo  
WHERE codigo = codlivro AND data >= '01/01/2000'
```

- Junção

```
SELECT titulo, data  
FROM livro JOIN emprestimo ON codigo = codlivro  
WHERE data >= '01/01/2000'
```

Renomeação de tabela

- Exemplo usando JOIN
 - Recuperar nome das pessoas e nomes das cidades onde elas moram

```
SELECT p.nome, c.nome
```

```
FROM pessoa p JOIN cidade c ON p.codcid = c.codigo
```

Renomeação de atributo

- Exemplo usando JOIN
 - Recuperar nome das pessoas e nomes das cidades onde elas moram

```
SELECT p.nome as nomePessoa, c.nome as nomeCidade  
FROM pessoa p JOIN cidade c ON p.codcid = c.codigo
```

Exemplos

- Recuperar título do livro, data de empréstimo e nome dos usuários, dos livros emprestados a partir de 2000
 - Produto cartesiano

```
SELECT l.titulo, e.data, u.nome
FROM livro l, emprestimo e, usuario u
WHERE l.codigo = e.codlivro AND e.codUsuario = u.codigo
        AND data >= '01/01/2000'
```

- Junção

```
SELECT titulo, data, nome
FROM livro l JOIN emprestimo e ON l.codigo = e.codlivro
        JOIN usuario u ON u.codigo = e.codUsuario
WHERE e.data >= '01/01/2000'
```

Tipos de Joins no SQL

- Como na álgebra, no SQL
 - **INNER JOINS**
 - TETA JOIN
 - NATURAL JOIN
 - **OUTER JOINS**
 - RIGHT JOIN
 - LEFT JOIN
 - FULL JOIN
-

TETA JOIN (JOIN)

```
SELECT m.nome, COUNT(c.data) AS qtdConsultas  
FROM medico m JOIN consulta c ON m.codigo = c.codmed  
WHERE m.idade > 30  
GROUP BY m.nome
```

NATURAL JOIN

```
SELECT m.nome, em.logradouro, em.cep  
FROM medico m NATURAL JOIN ender_medico em  
WHERE m.idade > 30
```

OUTER JOIN

```
SELECT m.nome, COUNT(c.data) AS qtdConsultas  
FROM medico m LEFT JOIN consulta c ON m.codigo = c.codmed  
WHERE m.idade > 30  
GROUP BY m.nome
```

```
SELECT m.nome, COUNT(c.data) AS qtdConsultas  
FROM medico m RIGHT JOIN consulta c ON m.codigo = c.codmed  
WHERE m.idade > 30  
GROUP BY m.nome
```

```
SELECT m.nome, COUNT(c.data) AS qtdConsultas  
FROM medico m FULL JOIN consulta c ON m.codigo = c.codmed  
WHERE m.idade > 30  
GROUP BY m.nome
```

OUTER JOIN

Anulando uma junção externa. Supondo o exemplo:

```
SELECT m.nome, c.data
```

```
FROM medico m LEFT OUTER JOIN consulta c ON m.codigo = c.codmed
```

<i>nome</i>	<i>data</i>
Ana	10/10/2009
Carlos	23/05/2009
Paula	04/05/2009
Everton	16/12/2009
Betania	NULL
Tito	NULL
Pedro	NULL
Laura	NULL

LEFT OUTER JOIN

Anulando uma junção externa

```
SELECT m.nome, c.data  
FROM medico m LEFT OUTER JOIN consulta c ON m.codigo = c.codmed  
WHERE data > '01/01/2009'
```

Filtra as linhas em cima da junção já feita

<i>nome</i>	<i>data</i>
Ana	10/10/2009
Carlos	23/05/2009
Paula	04/05/2009
Everton	16/12/2009
Betania	NULL
Tito	NULL
Pedro	NULL
Laura	NULL

LEFT OUTER JOIN

Anulando uma junção externa

```
SELECT m.nome, c.data  
FROM medico m LEFT OUTER JOIN consulta c ON m.codigo = c.codmed  
AND data > '01/01/2009'
```

O filtro é feito
durante a junção

<i>nome</i>	<i>data</i>
Ana	10/10/2009
Carlos	23/05/2009
Paula	04/05/2009
Everton	16/12/2009
Betania	NULL
Tito	NULL
Pedro	NULL
Laura	NULL

Exercícios



Exercícios

